Delivery Agentica: contratti ed esecuzione guidata.

Il framework per integrare agenti software nello sviluppo: principi, flusso, gate e controllo umano.

Premessa · Principi · Il flusso · Gate e controllo · Piattaforma · Adozione

Il punto di partenza

L'AI genera codice rapidamente, ma **senza vincoli formali accumula debito e allucinazioni**.

Uso spontaneo non governato

Cosa accade lasciando l'agente a briglia sciolta.

- **Prompt informali ed effimeri**
  I requisiti restano nella chat e si perdono alla chiusura del tab
- **Contratti impliciti violati**
  Il modello inventa campi o rompe la compatibilità OpenAPI
- **Review umana sfibrata**
  Pull request enormi con centinaia di righe non contestualizzate

Delivery con il framework

Un processo ingegneristico governato e riproducibile.

- **Specifiche e Use Case a monte**
  Artefatti formalizzati e versionati prima di toccare la tastiera
- **Gate deterministici**
  Compiler, linter, OpenAPI validation e test suite bloccano le deviazioni
- **Task atomici e immutabili**
  Diff compatti, circoscritti a un singolo Use Case e facili da verificare

I tre pilastri

Tre vincoli distinguono un esperimento isolato da un **processo ingegneristico ripetibile**.

01

Contratti prima del codice

Le specifiche (PRD, DR, OpenAPI, Use Case) sono la sorgente di verità, non i suggerimenti nei prompt.

02

Determinismo dove conta

Linter, compiler, contratti di schema e test automatici validano l'output: l'AI opera dentro binari rigidi.

03

Human-in-the-Loop strutturato

L'agente propone, compila bozze ed esegue; l'essere umano ha la titolarità delle decisioni e dei gate.

La regola guida: **l'AI non decide l'architettura**, realizza soluzioni dentro specifiche convalidate.

Pragmatismo e transizione

Non una rottura a tavolino, ma un ponte realistico: **continuità con i flussi attuali** e **minimo debito proprietario**.

Continuità operativa

Integrazione fluida, senza attriti di processo.

Integrazione nei flussi esistenti

Nessun cambio forzato di toolchain: gli agenti operano su Jira, Confluence, GitHub e CI già adottati dai team.

Transizione verso una delega crescente

Fase transitoria di affiancamento: si parte da compiti circoscritti per ampliare l'autonomia con la maturità dei modelli.

Minimalismo negli artefatti

Favorire il buy sul make ed evitare debito.

Protocolli ed ecosistema ancora fluidi

Skill, custom agent e protocolli non sono ancora stabilizzati: creare sovrastrutture proprietarie oggi è rischioso.

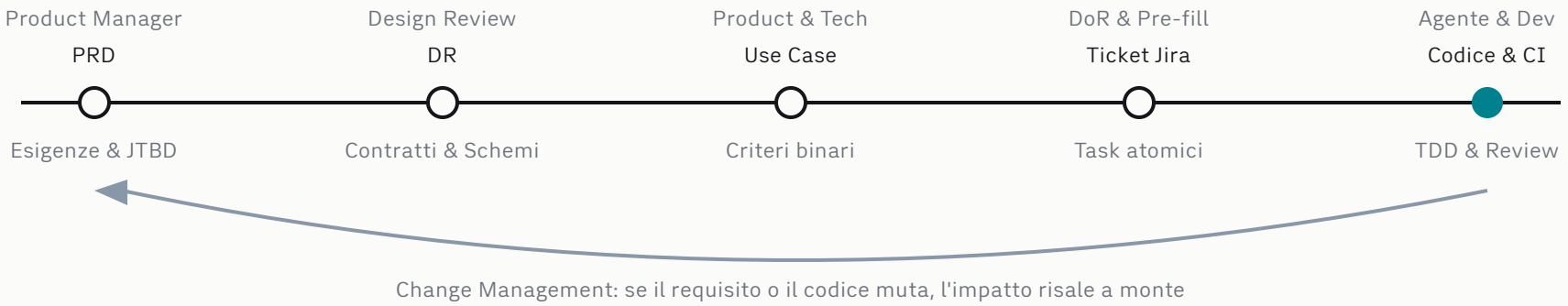
Scommessa sugli harness ufficiali

Gli strumenti di mercato integreranno ciò che oggi è ad hoc: sviluppiamo internamente solo lo stretto necessario.

L'obiettivo: **abilitare i team subito**, evitando di costruire codice custom destinato a diventare debito.

Il ciclo di delivery

Dall'esigenza di business al codice in esercizio attraverso **cinque stadi tracciabili**.



La catena del valore — ogni passaggio produce un contratto formale e testabile per lo stadio successivo.

Dai requisiti all'architettura

Il PRD circoscrive il problema; la Design Review stabilisce **contratti tecnici e decisioni irrevocabili**.

Specifiche di prodotto e architettura — APPROVAZIONE CONGIUNTA

PRD	**Definizione dell'esigenza** : Jobs To Be Done (JTBD), vincoli di business e perimetro escluso.
DESIGN REVIEW (DR)	**Modello architetturale** : flussi, modelli dati e interfacce esposte, redatti con template unificato.
DECISIONI DI DETTAGLIO	**Trade-off tecnici isolati** : decisioni specifiche discusse e approvate prima di confluire nella DR madre.
GATE 1: PLATFORM REVIEW	**Validazione vincolante** : nessun codice viene sviluppato prima che Product e Platform abbiano ratificato la DR.

Non esiste un documento monolitico: **le DR tematiche isolano il perimetro** e rendono i vincoli valutabili.

Contratti atomici per l'agente

Gli agenti falliscono sull'ambiguità: **gli Use Case traducono la DR in scenari binari e immutabili**.

Use Case e Criteri di Accettazione

Il contratto di comportamento per l'agente.

Formato Given / When / Then

Scenari nominali, casi limite e stati d'errore senza ambiguità verbale

Specchio dei test futuri

Ogni scenario dell'Use Case corrisponde direttamente a un test di accettazione

Perimetro chiuso

Un solo flusso logico verificabile in modo deterministico

Backlog Jira AI-Ready

La struttura operativa e atomica del lavoro.

Immutabilità della Story

Una volta approvata, la story non cambia in corsa: variazioni richiedono una CR

Definition of Ready (DoR)

L'agente accetta il ticket solo se contratti, dipendenze e schemi sono risolti

Shadow Pre-fill

L'agente suggerisce metadati e stime, l'ingegnere convalida con un clic

L'agente non deve interpretare: **la Definition of Ready garantisce l'assenza di presupposti oscuri**.

Sviluppo spec-first e TDD

L'agente implementa in un workspace isolato **partendo sempre dai contratti e dai test**.

01

Spec-first o Infer-and-Ratify

Nei progetti nuovi si parte da OpenAPI e Data Contract. Nei repository esistenti, l'agente analizza la codebase e propone un ADR locale per ratifica.

02

Test-Driven Development obbligatorio

L'agente traduce i criteri di accettazione dell'Use Case in test prima di scrivere il codice: il test rosso definisce l'obiettivo oggettivo.

03

Verifica locale con linter e CI isomorfa

L'agente compila, esegue la suite di test e corregge i fallimenti in autonomia prima di sottoporre qualsiasi modifica a review umana.

Il codice è valido solo quando **tutti i test di contratto e regressione passano** senza allentare le asserzioni.

Separazione delle responsabilità

Non deleghiamo la responsabilità: **l'ingegnere governa il sistema, l'agente accelera la stesura**.

Ingegnere e Product Manager

Direzione tecnica, giudizio e responsabilità.

Decisioni architetturali

Valuta i trade-off tecnici e sceglie i pattern di sistema

Definizione del valore

Stila i PRD, valida i JTBD e approva gli Use Case

Validazione e merge

Ispeziona le Pull Request, verifica i test e autorizza i rilasci

Agente software

Esecuzione guidata su compiti circoscritti e verificabili.

Sintesi e pre-fill

Compila bozze di DR, metadata Jira e matrici di tracciabilità

Spec-first generation

Produce client, modelli dati e stub conformi a schemi OpenAPI

TDD e refactoring

Scrive test derivati dagli Use Case e itera finché la suite è verde

Nessun agente dispone di **permessi autonomi di scrittura sui rami protetti**.

I cinque gate di controllo

Nessun artefatto avanza al livello successivo senza aver **superato un gate deterministico o umano**.

Gate 1 · Requisiti & Architettura

Verifica completezza PRD, DR e risoluzione dei punti aperti

UMANO · PRODUCT & PLATFORM

Gate 2 · Backlog & Ready for Dev

Verifica DoR su Jira, contratti allegati e Use Case atomici

AUTOMATICO + INGEGNERIA

Gate 3 · Contract & Security Gate

Validazione OpenAPI linter, filtri PII e assenza dati sensibili

AUTOMATICO DETERMINISTICO

Gate 4 · Code & Test Verification

CI verde, copertura test di regressione e review del diff su PR

UMANO · INGEGNERIA & QA

Gate 5 · Operations & Release

Checklist di readiness operativa, monitoraggio e piano di rollback

UMANO · OPS & PLATFORM

I controlli automatici filtrano le anomalie banali; **gli ingegneri presidiano l'intento e la sicurezza**.

Dove l'agente si ferma

L'autonomia è massima nella preparazione, ma **si arresta davanti a modifiche di stato o di codice condiviso**.

Definizione di priorità ed esigenze di business Decisione sul valore e perimetro del sistema	UMANO ESCLUSIVO
Pre-compilazione schede, stub e scenari Use Case Proposta di bozze strutturate a partire da linee guida	AGENTE AUTONOMO
Validazione DoR e approvazione presa in carico Verifica assenza di ambiguità prima del dispatch	UMANO OBBLIGATORIO
Scrittura codice e test nel workspace locale Ciclo red-green-refactor vincolato ai contratti di schema	AGENTE CONTROLLATO
Approvazione Pull Request e merge su main branch Verifica del diff, dei test di accettazione e adeguatezza tecnica	UMANO ESCLUSIVO
Rilascio e deploy in ambiente di produzione Pipeline CI/CD tracciata con autorizzazioni di servizio	PIPELINE PROTETTA

Nessun agente dispone di **token con privilegi amministrativi o di deploy diretto**.

Policy di accesso & Human-in-the-Loop

Skill, DAG e protocolli

Niente prompt casuali: gli agenti invocano **skill versionate e orchestrate come grafi aciclici**.

Architettura delle Skill

Capacità modulari, testabili e versionate.

Skill Registry centrale

Catalogo interno di compiti certificati (linter, parser OpenAPI, scaffolding)

Orchestrazione a DAG

Flussi di esecuzione composti come grafi diretti aciclici con input e output tipizzati

Dead Letter Queue (DLQ)

Se un'esecuzione stalla o supera i limiti, il contesto confluisce in una coda di audit

Protocolli e Integrazione

Confini netti tra ambiente locale e remoto.

Model Context Protocol (MCP)

Interfacce standardizzate verso Jira, Confluence e cataloghi dati di piattaforma

CLI native nel workspace

Compilatori, git e suite di test eseguiti direttamente nell'ambiente dell'ingegnere

Audit Log immutabile

Tracciamento delle chiamate di tool, prompt impiegati e token consumati

Ogni skill è corredata da **eval automatici prima di essere pubblicata** nel registry aziendale.

Zero trust per gli agenti

Gli agenti non accedono a dati reali e **ogni informazione in transito è filtrata preventivamente**.

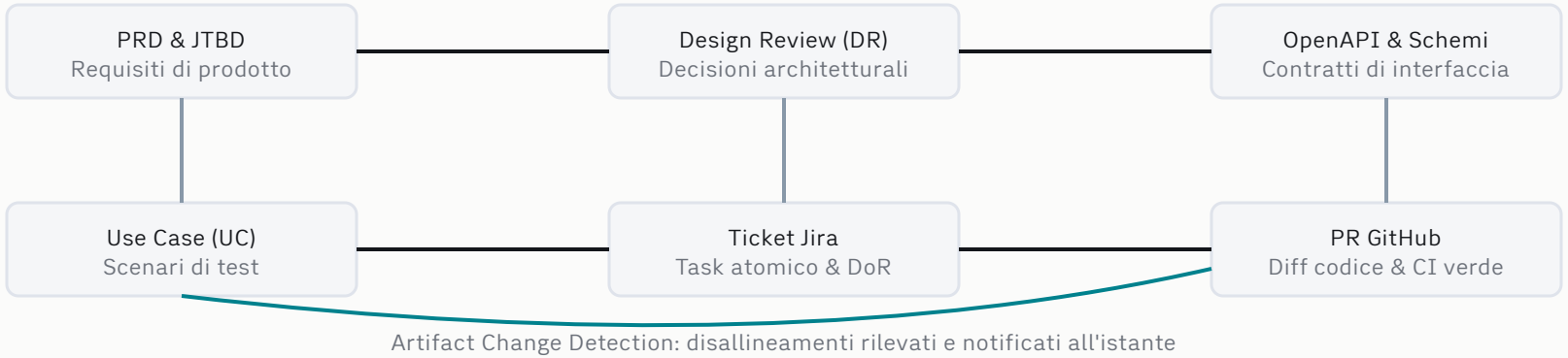
Modello di Sicurezza, Compliance e Privacy — ZERO TRUST

AGENT IAM	**Credenziali effimere a minimo privilegio** : token a scadenza rapida vincolati al singolo repository e alla specifica sessione.
FILTRO PII PRE-WRITE	**Mascheramento preventivo** : ispezione automatica dei payload prima di qualsiasi invocazione di modelli remoti.
ISOLAMENTO DATI	**Nessun dato di produzione** : gli agenti operano esclusivamente su fixture di test, mock o dati sintetici certificati.
SECURITY REVIEW AUTOMATIZZATA	**Analisi statica e SAST** : scansione vulnerabilità e verifica delle dipendenze integrate nella pipeline CI.

I dati personali e le informazioni sensibili o transazionali **non raggiungono mai il contesto dell'agente**.

Propagazione e Change Management

Ogni riga di codice è legata a una specifica: **se il contratto cambia, l'impatto si propaga in automatico**.



Matrice di propagazione — legami bidirezionali garantiscono che nessuna riga di codice resti orfana della specifica.

Adozione incrementale

Nessun passaggio monolitico: **validazione progressiva su progetti pilota con metriche oggettive**.

01

Fase 1 · Pilota controllato su team selezionati

Adozione dei template (PRD, DR, UC) e del backlog AI-ready su 1-2 servizi per calibrare i gate operativi.

02

Fase 2 · Misurazione e affinamento

Monitoraggio KPI: lead time, tasso di rifiuto PR, frequenza di intervento umano e feedback log degli agenti.

03

Fase 3 · Estensione a Platform

Consolidamento del registry delle skill aziendali, quality gate periodici e apertura progressiva a tutti i team.

La decisione richiesta a Platform è **l'autorizzazione alla fase pilota**, non l'imposizione universale immediata.

Domande?

Il framework non sostituisce gli ingegneri: **trasforma i requisiti in contratti e l'AI in un esecutore affidabile**.